

# Linux Embebido – Módulo 5

## Dispositivos de carácter (parte 2): **read/write** en profundidad e **ioctl**

Ing. Marín Ariel · Fabrizio Carlassara (BIDOC) ·

Pablo Gómez (BIDOC) · Alexis Hernández de la Cruz (BIS)

**Técnicas Digitales III**

Laboratorio de Sistemas Embebidos

Universidad Tecnológica Nacional

Facultad Regional Avellaneda

Departamento de Ingeniería Electrónica

# Índice

|                                                                 |    |
|-----------------------------------------------------------------|----|
| 1. Objetivos del módulo                                         | 3  |
| 2. Punto de partida: el driver del Módulo 4                     | 3  |
| 3. <b>read/write</b> en profundidad                             | 4  |
| 3.1. El rol de <code>off</code>                                 | 4  |
| 3.2. Buffers de tamaño arbitrario                               | 5  |
| 3.3. <code>ledbtn_read/ledbtn_write</code> actualizados         | 5  |
| 4. <b>ioctl</b> : qué problema resuelve                         | 6  |
| 4.1. La <code>syscall</code> y la interfaz en el driver         | 6  |
| 4.2. La codificación del número de comando                      | 7  |
| 4.3. Elegir el <i>magic number</i>                              | 8  |
| 5. Diseño de los comandos <b>ioctl</b> de <b>ledbtn</b>         | 8  |
| 5.1. El header compartido                                       | 8  |
| 5.2. Los cinco comandos                                         | 9  |
| 5.3. Implementación                                             | 10 |
| 6. Práctica: extender <code>/dev/ledbtn</code> con <b>ioctl</b> | 12 |
| 6.1. <code>ledbtn.h</code> : el header compartido               | 12 |
| 6.2. <code>ledbtn.c</code> : los cambios sobre el Módulo 4      | 12 |
| 6.3. <code>ledbtn_ctl.c</code> : el programa de control         | 12 |
| 6.4. Compilación, carga y prueba                                | 14 |
| 7. Cierre conceptual                                            | 14 |

## Presentación del módulo

El Módulo 4 dejó un driver de carácter funcional, `/dev/ledbtn`, pero con `read/write` resueltos de la forma más simple posible: un único byte, sin prestarle atención a `off` ni a buffers de más de un byte. Ese atajo fue deliberado – alcanzaba para la práctica de ese módulo – pero deja afuera dos piezas centrales de la interfaz real entre espacio de usuario y espacio de kernel. Este módulo cierra esas dos piezas, sobre el mismo driver y el mismo hardware, sin escribir nada de Device Tree nuevo. Primero, se generaliza `read/write` para que respeten `off` y acepten buffers de tamaño arbitrario. Segundo, y es el foco principal del módulo, se agrega `ioctl`: el mecanismo que usa Linux para todo lo que `read/write` no puede expresar bien – comandos puntuales y parámetros de configuración que no son “un flujo de bytes”. Se implementan cinco comandos que entre todos cubren las cuatro variantes de macro de `ioctl` (`_IO`, `_IOR`, `_IOW`, `_IOWR`), y como `echo/cat` no pueden invocar `ioctl()`, se escribe también un programa chico de espacio de usuario que sí lo hace.

## 1. Objetivos del módulo

Al finalizar este módulo, el alumno debería poder:

- Explicar el rol de `off` en `read/write` y por qué un driver debe actualizarlo si quiere soportar lecturas/escrituras parciales.
- Copiar datos de tamaño arbitrario entre espacio de usuario y espacio de kernel con `copy_to_user/copy_from_user`, respetando un buffer local acotado.
- Explicar qué problema resuelve `ioctl` que `read/write` no resuelven bien.
- Describir la codificación del número de comando de `ioctl` (dirección, tamaño, *magic number*, número de comando) y las macros `_IO/_IOR/_IOW/_IOWR` que la generan.
- Implementar `.unlocked_ioctl` en `struct file_operations`.
- Diseñar e implementar los comandos `ioctl` del driver `ledbtn`, cubriendo las cuatro variantes de macro.
- Escribir un programa de espacio de usuario en C que invoque `ioctl()` sobre `/dev/ledbtn`.

## 2. Punto de partida: el driver del Módulo 4

Este módulo no arranca de cero: extiende el mismo `ledbtn.c` del Módulo 4. Vale la pena repasar en una línea lo que ese driver ya tiene resuelto, porque no se vuelve a tocar acá:

- El registro completo del dispositivo de carácter (`alloc_chrdev_region`, `cdev_init/cdev_add`, `class_create/device_create`) – `ledbtn_init` y `ledbtn_exit` quedan **sin cambios**.

- El overlay de Device Tree `ledbtn-overlay.dts` y la obtención de los descriptores GPIO con `gpiod_get_from_of_node` – tampoco cambian.
- El montaje físico (LED en GPIO17, botón en GPIO27) – el mismo de la Práctica del Módulo 4.

Lo único que se modifica es el contenido de `ledbtn_read/ledbtn_write`, y se agrega una función nueva, `ledbtn_ioctl`, junto con la entrada correspondiente en `struct file_operations`.

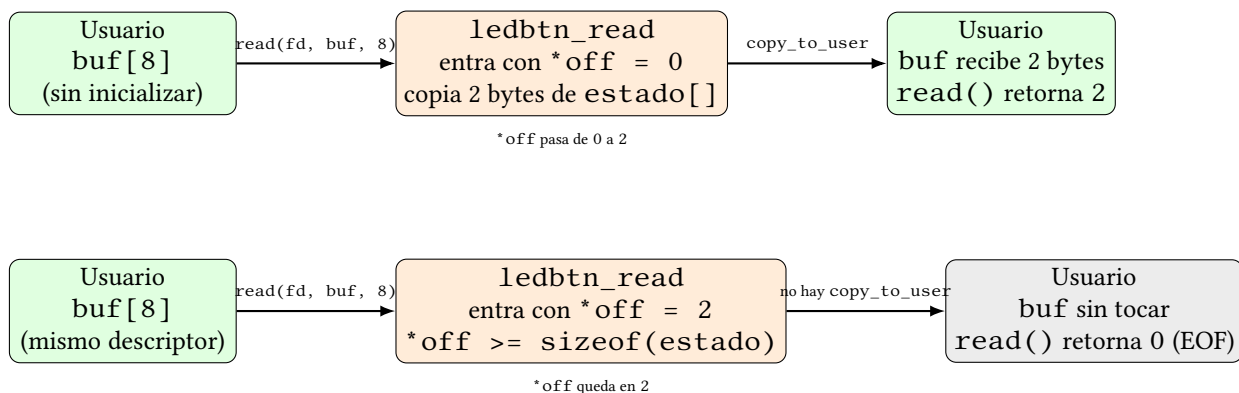
## 3. read/write en profundidad

### 3.1. El rol de `off`

`loff_t *off` es la posición actual dentro del archivo abierto – el mismo concepto que mueve `lseek`. El VFS se lo pasa al driver en cada llamada y espera que el driver lo **actualice** él mismo según cuántos bytes efectivamente transfirió; el VFS no lo hace por su cuenta [3]. Si un driver ignora `off` por completo, cada llamada a `read` vuelve a entregar el mismo dato desde el principio – un programa como `dd` que lee de a un byte por vez quedaría en un ciclo infinito, porque nunca ve el fin de archivo.

El Módulo 4 evitaba el problema por el lado fácil: como el dispositivo solo tenía un byte para entregar, alcanzaba con un chequeo binario (`if (*off > 0) return 0;`) para simular fin de archivo después de la primera lectura. Funciona, pero no generaliza: acá se lo reemplaza por el patrón correcto, que sirve para cualquier cantidad de bytes.

La Figura 1 sigue el rastro de `*off` y del contenido del buffer de usuario a lo largo de dos llamadas a `read()` **sobre el mismo descriptor de archivo** – por ejemplo, dos `read()` consecutivas que hace `cat` internamente hasta que una de ellas le devuelve 0.



**Figura 1:** Dos llamadas sucesivas a `read()` sobre el mismo `fd`. `*off` vive en el `struct file` del kernel – no en el buffer de usuario, y no se reinicia entre llamadas mientras el descriptor siga abierto –, así que la segunda llamada, aunque pide otra vez 8 bytes, no copia nada: `ledbtn_read` ya entregó todo lo que tenía. Si el driver no avanzara `*off` en la primera llamada, la segunda repetiría exactamente la misma copia en lugar de devolver fin de archivo.

## 3.2. Buffers de tamaño arbitrario

`len` es el tamaño del buffer de espacio de usuario, no una promesa de cuántos bytes hay que transferir – puede ser mayor o menor que lo que el driver tiene para dar. Un driver correcto nunca asume un `len` mínimo ni copia más de lo que `len` permite. Del lado del kernel, tampoco conviene reservar un buffer de tamaño arbitrario para cada `write`: alcanza con un buffer local pequeño y de tamaño fijo, y copiar como máximo lo que entra ahí [2, cap. 3].

## 3.3. `ledbtn_read/ledbtn_write` actualizados

El dispositivo pasa a representar su estado como texto: `read` devuelve `"0\n"` o `"1\n"` (dos bytes) en vez de un solo carácter, y `write` acepta `"0"/"1"/"on"/"off"`.

```
static ssize_t ledbtn_read(struct file *file, char __user *buf,
                          size_t len, loff_t *off)
{
    char estado[2];
    size_t restante;

    if (*off >= sizeof(estado))
        return 0; /* ya se entrego todo: fin de archivo */

    estado[0] = gpiod_get_value(boton_gpio) ? '1' : '0';
    estado[1] = '\n';

    restante = sizeof(estado) - *off;
    if (restante > len)
        restante = len;

    if (copy_to_user(buf, estado + *off, restante))
        return -EFAULT;

    *off += restante;
    return restante;
}

#define LEDBTN_BUF_MAX 8

static ssize_t ledbtn_write(struct file *file, const char __user *buf,
                           size_t len, loff_t *off)
{
    char local[LEDBTN_BUF_MAX];
    size_t n = len;
    int encender;

    if (n >= sizeof(local))
        n = sizeof(local) - 1; /* nunca copiar mas de lo que entra en local
    [] */
```

```

    if (copy_from_user(local, buf, n))
        return -EFAULT;
    local[n] = '\0';

    if (n > 0 && local[n - 1] == '\n')
        local[n - 1] = '\0'; /* recortar el salto de línea que agrega "echo"
                               */

    if (!strcmp(local, "1") || !strcmp(local, "on"))
        encender = 1;
    else if (!strcmp(local, "0") || !strcmp(local, "off"))
        encender = 0;
    else
        return -EINVAL;

    ledbtn_set_led(encender);

    return len; /* se consumio todo el buffer original */
}

```

#### Nota

`ledbtn_read` ahora sirve tanto a `cat` (que pide un buffer grande y se lleva los dos bytes de una vez) como a un lector que pida de a un byte – restante nunca excede ni `len` ni lo que falta por entregar, y `*off` avanza exactamente lo que se copió. `ledbtn_write` delega el encendido efectivo del LED en `ledbtn_set_led`, una función nueva que centraliza la lógica de polaridad – se explica en la Sección 5, porque también la usa `ioctl`.

## 4. ioctl: qué problema resuelve

`read/write` modelan un dispositivo como un flujo secuencial de bytes, y ese modelo alcanza para mucho. Pero no todo lo que un driver necesita exponer es “un dato que se lee o se escribe”: a veces es un **comando** puntual (reiniciar el dispositivo) o un **parámetro de configuración** que no tiene sentido mezclar con el flujo de datos normal (cambiar la polaridad con la que el driver interpreta sus propias escrituras). Forzar esos casos dentro de `write` – por ejemplo, inventando strings mágicos como “RESET” – funciona, pero mezcla dos cosas conceptualmente distintas y no tiene forma natural de devolver un resultado. `ioctl` es el mecanismo que Linux ofrece específicamente para esto [2, cap. 6].

### 4.1. La syscall y la interfaz en el driver

Desde espacio de usuario, la firma es `int ioctl(int fd, unsigned long cmd, ...)`: un descriptor de archivo, un número de comando, y un tercer argumento opcional cuyo

significado depende del comando (típicamente un puntero). Del lado del driver, el campo correspondiente en `struct file_operations` es:

```
long (*unlocked_ioctl)(struct file *file, unsigned int cmd, unsigned long arg);
```

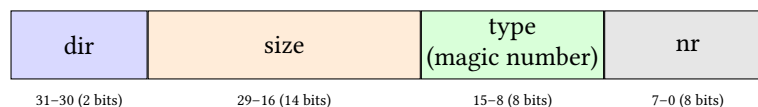
`arg` llega con el mismo valor que el tercer argumento de la syscall: si es un puntero, sigue siendo un puntero de **espacio de usuario**, y aplican las mismas reglas que en `read/write` – el driver no puede desreferenciarlo directamente, tiene que pasar por `copy_to_user/copy_from_user` [3].

### Nota

El nombre `unlocked_ioctl` es un resabio histórico: hasta el kernel 2.6.36 existía también un campo `ioctl` que el kernel invocaba *con* el Big Kernel Lock tomado. Ese campo desapareció; hoy `unlocked_ioctl` es el único que existe, y el prefijo “unlocked” ya no distingue nada – si el driver necesita exclusión mutua, es responsabilidad suya tomar su propio lock [3].

## 4.2. La codificación del número de comando

`cmd` no es un entero arbitrario: en Linux, por convención, empaqueta cuatro campos en sus 32 bits – la **dirección** de la transferencia de datos, su **tamaño**, un **magic number** que identifica al driver, y un **número de comando** dentro de ese driver. Esa convención evita que dos drivers distintos elijan por casualidad el mismo `cmd` para cosas distintas [2, cap. 6].



**Figura 2: Codificación de 32 bits del número de comando `cmd`, de bit más significativo (izquierda) a menos significativo (derecha). `dir` vale `_IOC_NONE`, `_IOC_WRITE`, `_IOC_READ` o la combinación de las dos últimas.**

Las macros de `<linux/ioctl.h>` arman ese entero automáticamente, así que en la práctica nunca se manipulan los bits a mano:

- `_IO(type, nr)`: comando sin transferencia de datos.
- `_IOR(type, nr, datatype)`: el driver **lee** información hacia espacio de usuario (dirección `_IOC_READ`); `datatype` solo se usa para calcular el campo `size` con `sizeof`, nunca se declara una variable de ese tipo.
- `_IOW(type, nr, datatype)`: espacio de usuario **escribe** datos hacia el driver (`_IOC_WRITE`).
- `_IOWR(type, nr, datatype)`: las dos direcciones en la misma llamada.

**Nota**

La definición exacta de estas macros – y de `_IOC_NRBITS`, `_IOC_TYPEBITS`, etc. que fijan el ancho de cada campo – está en `include/uapi/asm-generic/ioctl.h` del código fuente del kernel; se puede navegar con Elixir [5] igual que se hizo con las firmas de `class_create` en el Módulo 4.

### 4.3. Elegir el *magic number*

`type` (el “magic number”) identifica al driver dentro del espacio de comandos de todo el sistema. Se elige un valor que otros drivers no usen, y la lista de valores ya tomados está documentada – para este driver conviene revisarla antes de fijar el propio [4]. En `ledbtn` se usa el carácter `'1'`.

## 5. Diseño de los comandos `ioctl` de `ledbtn`

### 5.1. El header compartido

Los comandos se definen una sola vez, en un header que incluyen tanto el driver (`ledbtn.c`) como el programa de espacio de usuario que los invoca – así ambos lados están de acuerdo con la codificación exacta de cada `cmd`:

```
#ifndef _LEDBTN_H_
#define _LEDBTN_H_

#include <linux/ioctl.h>

#define LEDBTN_MAGIC '1'

#define LEDBTN_RESET _IO(LEDBTN_MAGIC, 0)
#define LEDBTN_GET_BOTON _IOR(LEDBTN_MAGIC, 1, int)
#define LEDBTN_SET_POLARIDAD _IOW(LEDBTN_MAGIC, 2, int)
#define LEDBTN_SET_LED _IOW(LEDBTN_MAGIC, 3, int)
#define LEDBTN_XPOLARIDAD _IOWR(LEDBTN_MAGIC, 4, int)

#endif
```

`<linux/ioctl.h>` existe tanto en el árbol del kernel como en las cabeceras de `glibc` para espacio de usuario, con las mismas macros – por eso el mismo header compila de los dos lados sin cambios.

## 5.2. Los cinco comandos

| Comando                           | Macro              | Dirección      | Qué hace                                                                                                                                 |
|-----------------------------------|--------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <code>LEDBTN_RESET</code>         | <code>_IO</code>   | ninguna        | Apaga el LED sin pasar por <code>write</code> .                                                                                          |
| <code>LEDBTN_GET_BOTON</code>     | <code>_IOR</code>  | kernel→usuario | Lee el estado del botón vía <code>ioctl</code> , como alternativa a <code>read</code> .                                                  |
| <code>LEDBTN_SET_POLARIDAD</code> | <code>_IOW</code>  | usuario→kernel | Recibe un <code>int</code> : si es distinto de 0, invierte qué valor de GPIO corresponde a “LED encendido” para las próximas escrituras. |
| <code>LEDBTN_SET_LED</code>       | <code>_IOW</code>  | usuario→kernel | Recibe un <code>int</code> (0/1) y prende o apaga el LED directamente, como alternativa a <code>write</code> .                           |
| <code>LEDBTN_XPOLARIDAD</code>    | <code>_IOWR</code> | bidireccional  | Fija la nueva polaridad y devuelve la anterior, en la misma llamada.                                                                     |

`LEDBTN_RESET`, `LEDBTN_GET_BOTON` y `LEDBTN_SET_LED` son la contraparte directa de lo que ya hacían `write` y `read`: quedan acá para mostrar que la misma operación puede exponerse por las dos vías, y para que quede claro que `ioctl` no reemplaza a `read/write` sino que cubre lo que a ellos les queda incómodo. La diferencia entre `LEDBTN_SET_LED` y `write` es la misma que entre `LEDBTN_GET_BOTON` y `read`: `write` recibe y parsea texto ("on"/"1"/...), mientras que `LEDBTN_SET_LED` recibe directamente el `int` binario que espera `ledbtn_set_led` – sin la vuelta de convertir un número a string de un lado y parsearlo de nuevo del otro. `LEDBTN_SET_POLARIDAD` y `LEDBTN_XPOLARIDAD` son el ejemplo pedido originalmente, “configurar el driver desde espacio de usuario” sin que esa configuración conviva con el flujo de datos de `write`; por eso hay dos comandos `_IOW` en esta lista y no uno solo – `LEDBTN_SET_LED` controla el hardware directamente, `LEDBTN_SET_POLARIDAD` configura *cómo* el driver interpreta esos controles.

**Nota**

**LEDBTN\_XPOLARIDAD, el patrón “exchange”.** Es habitual que un `ioctl` que fija un parámetro también convenga que devuelva el valor anterior, para que el programa de usuario pueda restaurarlo después sin tener que llevarse la cuenta por su cuenta. El driver `scull` de LDD3 tiene exactamente este patrón con `IOCXQUANTUM` [2, cap. 6]: el mismo puntero de `arg` se usa primero para leer el valor nuevo (`copy_from_user`) y después para escribir el valor viejo en el mismo lugar (`copy_to_user`) – desde el punto de vista del programa de usuario, la variable que pasó por referencia “entra” con un valor y “sale” con otro, en la misma llamada.

### 5.3. Implementación

```
static void ledbtn_set_led(int encender)
{
    gpiod_set_value(led_gpio, polaridad_invertida ? !encender : encender);
}

static long ledbtn_ioctl(struct file *file, unsigned int cmd, unsigned
    long arg)
{
    int __user *argp = (int __user *)arg;
    int valor, anterior;

    switch (cmd) {
    case LEDBTN_RESET:
        ledbtn_set_led(0);
        break;

    case LEDBTN_GET_BOTON:
        valor = gpiod_get_value(boton_gpio);
        if (copy_to_user(argp, &valor, sizeof(valor)))
            return -EFAULT;
        break;

    case LEDBTN_SET_POLARIDAD:
        if (copy_from_user(&valor, argp, sizeof(valor)))
            return -EFAULT;
        polaridad_invertida = !!valor;
        break;

    case LEDBTN_SET_LED:
        if (copy_from_user(&valor, argp, sizeof(valor)))
            return -EFAULT;
        ledbtn_set_led(!!valor);
        break;
    }
```

```

case LEDBTN_XPOLARIDAD:
    if (copy_from_user(&valor, argp, sizeof(valor)))
        return -EFAULT;
    anterior = polaridad_invertida;
    polaridad_invertida = !!valor;
    if (copy_to_user(argp, &anterior, sizeof(anterior)))
        return -EFAULT;
    break;

default:
    return -ENOTTY; /* comando no reconocido: convencion POSIX */
}

return 0;
}

```

`polaridad_invertida` es una variable `bool` estática nueva (arranca en `false`), declarada igual que `led_gpio` y `boton_gpio` en el Módulo 4. `-ENOTTY` es el código de error que la convención del kernel usa para “`ioctl` no reconocido” – un nombre histórico poco intuitivo (viene de los drivers de terminal), pero es el que `glibc` y las herramientas esperan [2, cap. 6].

#### Nota

**Por qué `!!valor` y no `valor` directo.** `valor` llega desde espacio de usuario vía `copy_from_user` y puede ser cualquier `int` – 2, -5, 1000 –, no necesariamente 0 o 1. `!` es la negación lógica: vale 1 si el operando es 0, y 0 en cualquier otro caso. Aplicarla dos veces (`!!valor`) hace el camino inverso y colapsa *cualquier* valor distinto de cero a exactamente 1, dejando 0 sin cambios – el resultado queda siempre en `{0, 1}`, sin importar qué mandó el programa de usuario. En este driver, como `polaridad_invertida` ya está declarada `bool`, la conversión a 0/1 ocurre igual con una asignación directa (`polaridad_invertida = valor;`), porque el propio lenguaje normaliza cualquier valor distinto de cero a 1 al convertirlo a `bool`. El modismo `!!` se usa de todas formas porque es la forma habitual en el kernel de normalizar un entero a booleano **de forma explícita** – deja la intención clara con solo leer la línea, y sigue funcionando igual de bien si mañana la variable pasa a ser un `int` en lugar de un `bool`. `ledbtn_set_led` recibe el mismo tratamiento en el caso `LEDBTN_SET_LED`, donde importa un poco más: su parámetro `encender` es un `int` común, no un `bool`, y cuando `polaridad_invertida` es falsa la función lo pasa **tal cual** a `gpiod_set_value` sin normalizarlo – pasarle ya un 0/ 1 desde el llamador es más prolijo que confiar en que `gpiod_set_value` trate cualquier valor no nulo como “encendido”.

Por último, se agrega la entrada correspondiente a `struct file_operations`:

```
static const struct file_operations ledbtn_fops = {
    .owner = THIS_MODULE,
    .open = ledbtn_open,
    .release = ledbtn_release,
    .read = ledbtn_read,
    .write = ledbtn_write,
    .unlocked_ioctl = ledbtn_ioctl,
};
```

## 6. Práctica: extender /dev/ledbtn con ioctl

El hardware y el overlay son los mismos del Módulo 4 (Sección 2): no hay nada que volver a cablear ni a compilar del lado del Device Tree.

### 6.1. ledbtn.h: el header compartido

El de la Sección 5, en un archivo aparte junto a ledbtn.c.

### 6.2. ledbtn.c: los cambios sobre el Módulo 4

El archivo suma dos `#include` nuevos, para `strcmp` y para el header propio (`<linux/string.h>` y `"ledbtn.h"`), la variable `polaridad_invertida`, y las funciones `ledbtn_set_led` y `ledbtn_ioctl` de la Sección 5. También reemplaza `ledbtn_read/ledbtn_write` por las versiones de la Sección 3. El resto – `ledbtn_open`, `ledbtn_release`, `ledbtn_init`, `ledbtn_exit` y el `Makefile` – queda exactamente como en el Módulo 4.

### 6.3. ledbtn\_ctl.c: el programa de control

`echo/cat` no tienen forma de invocar `ioctl()` – son programas genéricos que solo llaman `read/write/open/close`. Probar los comandos nuevos requiere un programa propio:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include "ledbtn.h"

static void uso(const char *argv0)
{
    fprintf(stderr,
        "Uso: %s reset|get-boton|set-led <0|1>|set-pol <0|1>|x-pol <0|1>\n",
        argv0);
}
```

```
    exit(1);
}

int main(int argc, char *argv[])
{
    int fd, valor;

    if (argc < 2)
        uso(argv[0]);

    fd = open("/dev/ledbtn", O_RDWR);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    if (!strcmp(argv[1], "reset")) {
        if (ioctl(fd, LEDBTN_RESET) < 0)
            perror("ioctl LEDBTN_RESET");
    } else if (!strcmp(argv[1], "get-boton")) {
        if (ioctl(fd, LEDBTN_GET_BOTON, &valor) < 0)
            perror("ioctl LEDBTN_GET_BOTON");
        else
            printf("boton: %d\n", valor);
    } else if (!strcmp(argv[1], "set-led") && argc == 3) {
        valor = atoi(argv[2]);
        if (ioctl(fd, LEDBTN_SET_LED, &valor) < 0)
            perror("ioctl LEDBTN_SET_LED");
    } else if (!strcmp(argv[1], "set-pol") && argc == 3) {
        valor = atoi(argv[2]);
        if (ioctl(fd, LEDBTN_SET_POLARIDAD, &valor) < 0)
            perror("ioctl LEDBTN_SET_POLARIDAD");
    } else if (!strcmp(argv[1], "x-pol") && argc == 3) {
        valor = atoi(argv[2]);
        if (ioctl(fd, LEDBTN_XPOLARIDAD, &valor) < 0)
            perror("ioctl LEDBTN_XPOLARIDAD");
        else
            printf("polaridad anterior: %d\n", valor); /* valor fue
                sobreescrito */
    } else {
        uso(argv[0]);
    }
}
```

```
close(fd);
return 0;
}
```

### Nota

El caso `x-pol` hace visible el patrón “exchange” en el lado de usuario: `valor` se pasa por dirección (`&valor`) con la polaridad **nueva** adentro, y después de que `ioctl()` retorna, esa misma variable quedó **sobreescrita** con la polaridad **anterior** – por eso el `printf` de después imprime lo que `valor` tiene en ese momento, no lo que el usuario pasó por `argv`.

## 6.4. Compilación, carga y prueba

```
# Compilar el modulo (igual que en el Modulo 4)
make

# Compilar el programa de control
gcc -Wall -o ledbtn_ctl ledbtn_ctl.c

# Cargar el modulo (el overlay ya debe estar aplicado)
sudo insmod ledbtn.ko

# read/write extendidos: ahora tambien aceptan "on"/"off"
echo "on" | sudo tee /dev/ledbtn
echo "off" | sudo tee /dev/ledbtn
cat /dev/ledbtn # imprime "0" o "1" seguido de salto de linea

# Los cinco comandos ioctl
sudo ./ledbtn_ctl reset
sudo ./ledbtn_ctl get-boton
sudo ./ledbtn_ctl set-led 1 # prende el LED sin pasar por write
sudo ./ledbtn_ctl set-pol 1 # invierte la polaridad
sudo ./ledbtn_ctl set-led 1 # con la polaridad invertida, esto apaga el
    LED
sudo ./ledbtn_ctl x-pol 0 # vuelve a la polaridad normal, imprime la
    anterior

# Descargar el modulo
sudo rmmod ledbtn
```

## 7. Cierre conceptual

Este módulo terminó de completar la interfaz de `/dev/ledbtn` que el Módulo 4 dejó abierta a propósito. Los puntos clave para retener:

- Un driver correcto respeta `off` y el `len` que le pasan `read/write`: nunca asume un tamaño de buffer ni ignora la posición dentro del archivo.
- `ioctl` existe para lo que `read/write` no modelan bien: comandos puntuales y configuración, no un flujo de datos.
- El número de comando de `ioctl` codifica dirección, tamaño, *magic number* y número de comando; las macros `_IO/_IOR/_IOW/_IOWR` lo arman de forma segura, sin manipular bits a mano.
- El patrón “exchange” (`_IOWR` que fija un valor nuevo y devuelve el anterior en la misma llamada) es una técnica real, no un capricho del driver de este curso – aparece en el propio `scull` de LDD3.

Con esto, `ledbtn` queda con una interfaz completa: `read/write` generalizados e `ioctl` para lo que no encaja ahí. El Módulo 6 cambia de eje: en lugar de seguir extendiendo este driver inventado, reemplaza la búsqueda manual de `of_find_compatible_node` por el modelo real de `platform_driver` con `probe()`, aplicado a un periférico existente – un UART secundario de la Raspberry Pi – que el Módulo 7 usa para la comunicación real con la placa ESP32-S3.

## Referencias

- [1] Vasquez, F. y Simmonds, C. (2021). *Mastering Embedded Linux Programming*, 3.<sup>a</sup> edición. Packt Publishing.
- Cap. 11, *Interfacing with Device Drivers* – panorama de las distintas formas en que Linux expone hardware a espacio de usuario, incluyendo dispositivos de carácter.
- [2] Corbet, J., Rubini, A. y Kroah-Hartman, G. (2005). *Linux Device Drivers*, 3.<sup>a</sup> edición. O’Reilly Media. Disponible libremente en <https://lwn.net/Kernel/LDD3/>.
- Cap. 3, *Char Drivers* – `copy_to_user/copy_from_user` y el contrato de buffers de tamaño arbitrario.
  - Cap. 6, *Advanced Char Driver Operations* – `ioctl` en profundidad: number encoding, macros `_IO/_IOR/_IOW/_IOWR`, y el patrón “exchange” de `scull` (`IOCXQUANTUM`) del que se toma `LEDBTN_XPOLARIDAD`.
- [3] The Linux Kernel development community. *Overview of the Linux Virtual File System*. <https://docs.kernel.org/filesystems/vfs.html>. Describe `struct file_operations` campo por campo – incluido `.unlocked_ioctl` – y el contrato de `off` y de los punteros de espacio de usuario en `read/write/ioctl`.
- [4] The Linux Kernel development community. *Ioctl Number Definitions*. <https://docs.kernel.org/userspace-api/ioctl/ioctl-number.html>.

Convención de asignación de *magic numbers* para `ioctl` y lista de los ya tomados por otros subsistemas del kernel, para evitar colisiones.

- [5] Bootlin. *Elixir Cross Referencer – Linux source code*. <https://elixir.bootlin.com/linux/latest/source>. Herramienta de navegación del código fuente del kernel de Linux con referencias cruzadas: útil para verificar la definición exacta de las macros `_IO/_IOR/_IOW/_IOWR` en `include/uapi/asm-generic/ioctl.h`.